

# HMM Viterbi POS Tagger

Hidden Markov Model + Viterbi Algorithm for Part-of-Speech Tagging

**90.5%**

Test Accuracy

**283**

Test Tokens

**21**

POS Tags

Generated: March 10, 2026

**Part-of-Speech (POS) tagging** assigns a grammatical category (noun, verb, adjective ...) to every word in a sentence.

## Example

| Token | The | cat | sat | on | the | mat | . |
|-------|-----|-----|-----|----|-----|-----|---|
| Tag   | DT  | NN  | VBD | IN | DT  | NN  | . |

## Why does it matter?

- Foundation for higher-level NLP: parsing, NER, machine translation, QA
- Penn Treebank tags used here – 21-tag subset

## Challenge

- Words are ambiguous: 'run' can be a verb (VB) or noun (NN)
- Context determines the correct tag

A **first-order Hidden Markov Model (HMM)** treats POS tags as hidden states and words as observations.

### Two core independence assumptions

- **Markov assumption:** the current tag depends only on the previous tag, not the full history.

$$\blacksquare P(t_i | t_1 \dots t_{i-1}) = P(t_i | t_{i-1})$$

- **Output independence:** the word depends only on its own tag.

$$\blacksquare P(w_i | w_1 \dots w_n, t_1 \dots t_n) = P(w_i | t_i)$$

### Joint probability of a sentence

$$P(w_{1:n}, t_{1:n}) = P(t_1 | \text{START}) \times \text{prod } P(t_i | t_{i-1}) \times \text{prod } P(w_i | t_i)$$

### Parameter estimation

- Maximum-Likelihood Estimation (MLE) from labelled training data
- Laplace (add- $\alpha$ ) smoothing ( $\alpha = 1.0$ ) to handle unseen events and out-of-vocabulary words

| Parameter              | Symbol                            | Formula (smoothed)                                                                                                    |
|------------------------|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Initial probability    | $\pi(t) = P(t \mid \text{START})$ | $(\text{Count}(\text{START} \rightarrow t) + \alpha) / (\text{Count}(\text{START} \rightarrow *) + \alpha \cdot  T )$ |
| Transition probability | $A(t_i, t_j) = P(t_j \mid t_i)$   | $(\text{Count}(t_i \rightarrow t_j) + \alpha) / (\text{Count}(t_i \rightarrow *) + \alpha \cdot  T )$                 |
| Emission probability   | $B(t, w) = P(w \mid t)$           | $(\text{Count}(t, w) + \alpha) / (\text{Count}(t) + \alpha \cdot ( V  + 1))$                                          |

### Laplace (add- $\alpha$ ) smoothing — why it matters

- Prevents zero probabilities for unseen word–tag pairs
- Out-of-vocabulary words mapped to a special token
- $\alpha = 1.0$  adds one pseudo-count to every possible event

### Corpus statistics

| 350 training sentences | 827 unique words    | 21 POS tags     | 50 test sentences |
|------------------------|---------------------|-----------------|-------------------|
| Manually labelled      | Penn Treebank style | Including <UNK> | 283 test tokens   |

Given a trained HMM, find the **most-probable tag sequence** for an input sentence using dynamic programming.

### Recurrence (log-space to avoid underflow)

$$\delta_t(j) = \max_i [ \delta_{t-1}(i) + \log A(i, j) ] + \log B(j, w_t)$$

### Three passes

- **Initialisation** ( $t = 0$ ):  $\delta_0(j) = \log \pi(j) + \log B(j, w_0)$
- **Recursion** ( $t = 1 \dots n-1$ ): fill the Viterbi table and store backpointers
- **Backtrace**: recover the optimal path by following backpointers from the best final state

### Complexity

| Time                | Space             | Exact?                 |
|---------------------|-------------------|------------------------|
| $O(n \times  T ^2)$ | $O(n \times  T )$ | Yes – globally optimal |

| Stage                  | Module                                                                     | What it does                                                                                     |
|------------------------|----------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| 1 – Train              | src/train.py<br>src/hmm_model.py                                           | Parse word/TAG corpus, count transitions & emissions, compute smoothed log-probabilities.        |
| 2 – Decode             | src/viterbi.py                                                             | Run Viterbi on each test sentence to find the most-probable tag sequence in $O(n \cdot  T ^2)$ . |
| 3 – Evaluate           | src/evaluate.py                                                            | Token accuracy, per-tag precision / recall / F1, confusion matrix.                               |
| 4 – Visualise & Report | src/visualize.py<br>report/generate_report.py<br>report/generate_slides.py | 5 analysis figures (heat-maps, bar charts) + PDF report + this slide deck.                       |

Run the full pipeline with a single command:

```
python main.py
```

Penn Treebank–style word/TAG corpus. **350 training sentences** | **50 test sentences** (no overlap).

| Tag | Description                       |
|-----|-----------------------------------|
| .   | Sentence-final punctuation        |
| CC  | Coordinating conjunction          |
| CD  | Cardinal number                   |
| DT  | Determiner                        |
| IN  | Preposition / subordinating conj. |
| JJ  | Adjective                         |
| JJR | Adjective, comparative            |
| NN  | Noun, singular                    |
| NNP | Proper noun, singular             |
| NNS | Noun, plural                      |
| PRP | Personal pronoun                  |

| Tag   | Description                       |
|-------|-----------------------------------|
| PRP\$ | Possessive pronoun                |
| RB    | Adverb                            |
| RP    | Particle                          |
| TO    | Infinitival to                    |
| VB    | Verb, base form                   |
| VBD   | Verb, past tense                  |
| VBG   | Verb, gerund                      |
| VBN   | Verb, past participle             |
| VBP   | Verb, present non-3rd             |
| VBZ   | Verb, present 3rd-person singular |
|       |                                   |

# 90.46%

token-level accuracy on the held-out test set

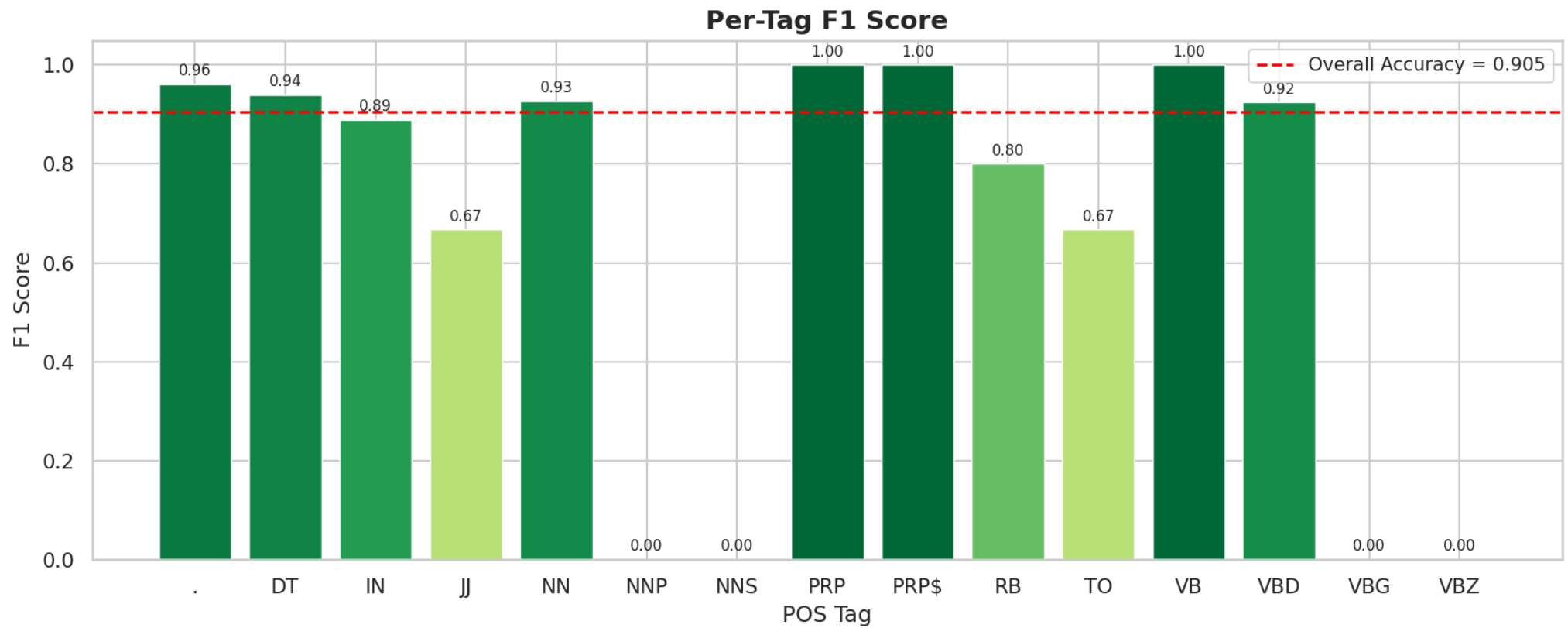
| Metric             | Value |
|--------------------|-------|
| Total tokens       | 283   |
| Correctly tagged   | 256   |
| Incorrectly tagged | 27    |

*Trained from **scratch** using only counts and Laplace smoothing — no neural networks, no pre-trained embeddings.*

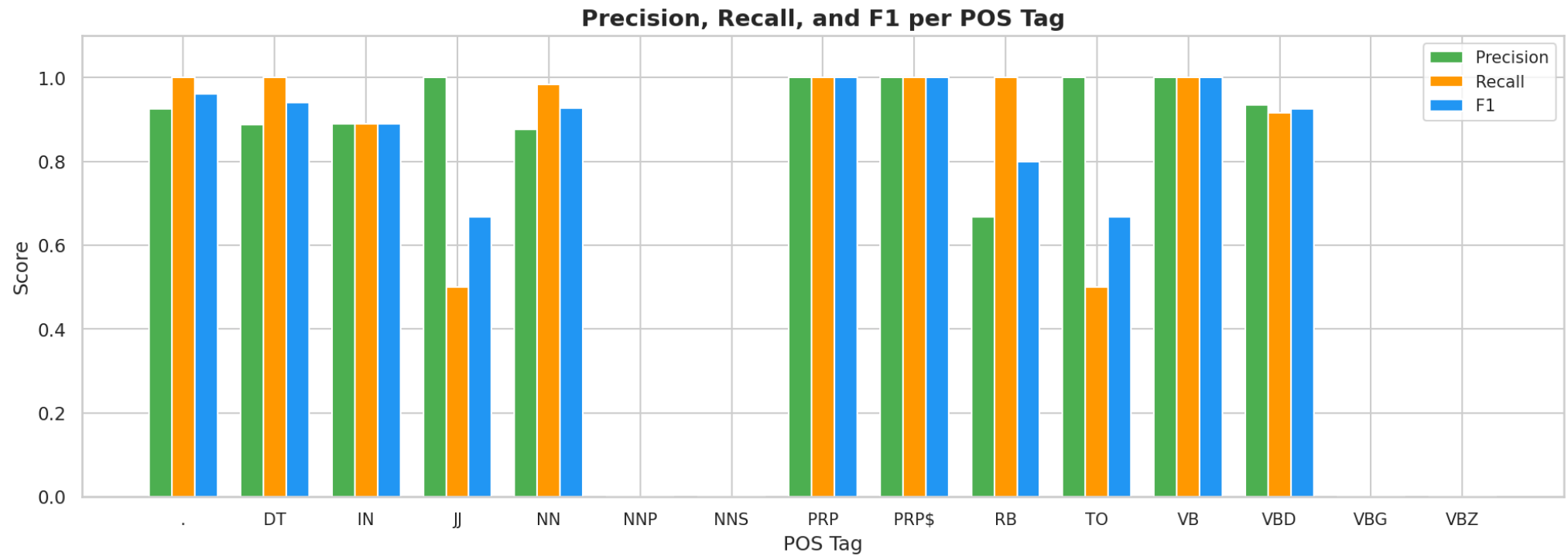


| Tag | Precision | Recall | F1     | Tag               | Precision     | Recall        | F1            |
|-----|-----------|--------|--------|-------------------|---------------|---------------|---------------|
| .   | 0.9259    | 1.0000 | 0.9615 | PRP\$             | 1.0000        | 1.0000        | 1.0000        |
| DT  | 0.8871    | 1.0000 | 0.9402 | RB                | 0.6667        | 1.0000        | 0.8000        |
| IN  | 0.8889    | 0.8889 | 0.8889 | TO                | 1.0000        | 0.5000        | 0.6667        |
| JJ  | 1.0000    | 0.5000 | 0.6667 | VB                | 1.0000        | 1.0000        | 1.0000        |
| NN  | 0.8769    | 0.9828 | 0.9268 | VBD               | 0.9348        | 0.9149        | 0.9247        |
| NNP | 0.0000    | 0.0000 | 0.0000 | VBG               | 0.0000        | 0.0000        | 0.0000        |
| NNS | 0.0000    | 0.0000 | 0.0000 | VBZ               | 0.0000        | 0.0000        | 0.0000        |
| PRP | 1.0000    | 1.0000 | 1.0000 | <b>Macro avg.</b> | <b>0.6787</b> | <b>0.6524</b> | <b>0.6517</b> |

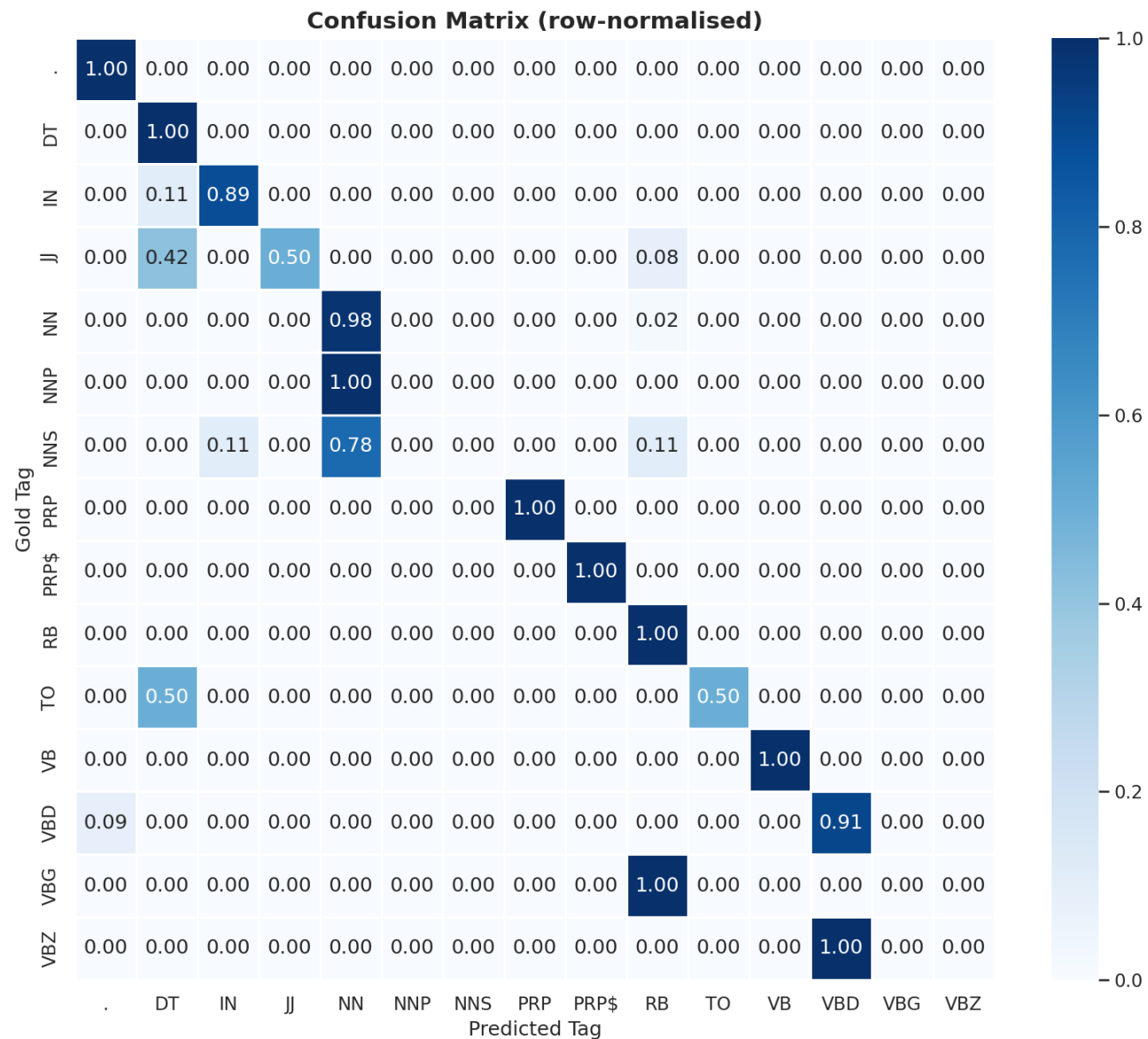
**Note:** Tags with 0.0 precision / recall / F1 (NNP, NNS, VBG, VBZ) do not appear in the test set in sufficient quantity for reliable estimation.



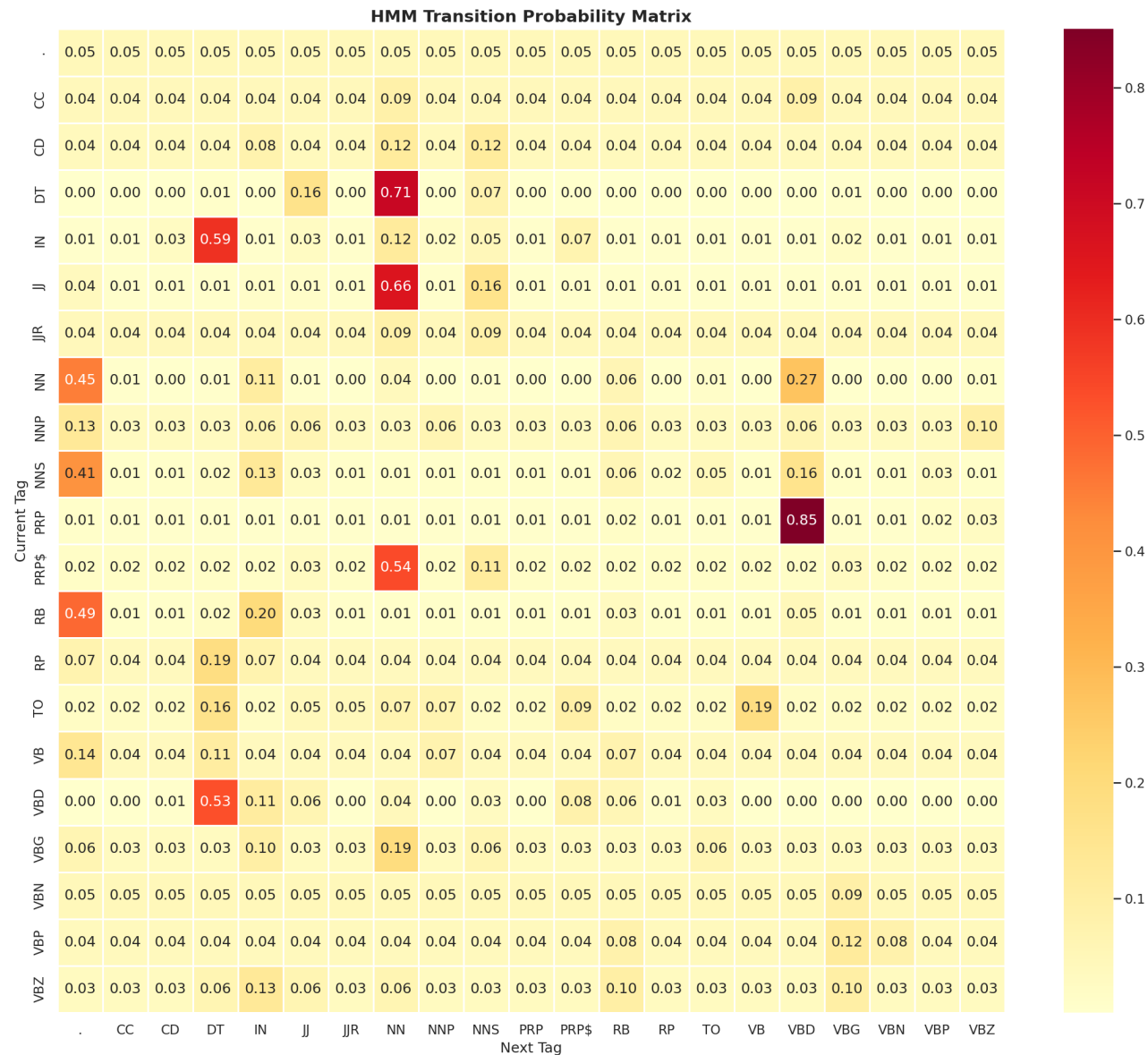
*Per-Tag F1 Score. Bars are coloured red (low) to green (high). Dashed line marks overall token accuracy.*



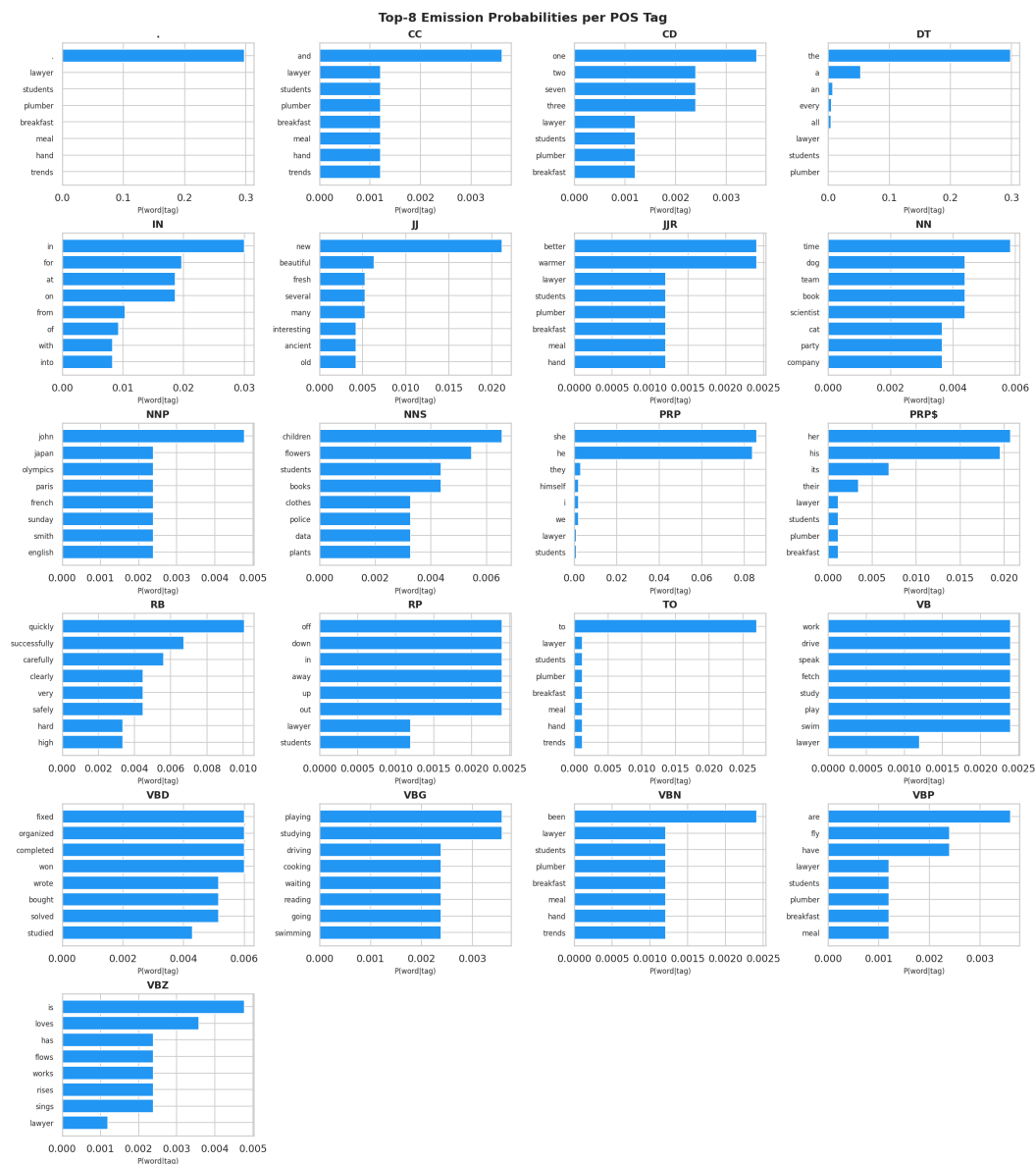
*Precision, Recall, and F1 per POS Tag. Grouped bars allow direct metric comparison.*



Row-normalised confusion matrix. Diagonal = per-tag recall. Off-diagonal cells reveal systematic confusions.



HMM Transition Probability Matrix. Cell (i,j) = smoothed  $P(\text{tag}_j \mid \text{tag}_i)$ .



Top-8 emission probabilities per tag. Each sub-plot shows the words most associated with a given POS.

### Strengths

- **90.46%** token accuracy — strong baseline from a simple statistical model with no neural components.
- Perfect F1 = 1.00 for: **PRP**, **PRP\$**, **VB**. These tags have highly distinctive vocabulary.
- Log-space Viterbi is numerically stable even for long sentences.

### Limitations & error sources

- Common confusions: NN vs NNP (singular vs. proper noun), VBD vs VBN (past tense vs. past participle). Both pairs share lexical forms.
- Tags NNP, NNS, VBG, VBZ score F1 = 0.00 — rare in the small test set.
- First-order Markov assumption: only the immediately previous tag is considered. Longer-range syntax is ignored.
- Small corpus (350 training sentences) limits coverage.

### Key take-aways

- A first-order HMM + Viterbi is a complete, interpretable POS tagger achievable in pure Python from scratch.
- Laplace smoothing is essential for robustness to unseen words.
- The Viterbi algorithm guarantees the globally optimal tag sequence in polynomial time.

### Possible extensions

| Extension                                             | Expected benefit                               |
|-------------------------------------------------------|------------------------------------------------|
| Higher-order HMM (bigram/trigram tags)                | Captures longer-range syntactic dependencies   |
| Morphological OOV features (suffixes, capitalisation) | Better handling of unknown words               |
| Discriminative model (CRF, BiLSTM-CRF)                | State-of-the-art accuracy (~97–98%)            |
| Pre-trained language model fine-tuning (BERT)         | Near-human performance on standard benchmarks  |
| Beam search decoding                                  | Faster approximate decoding for large tag sets |